

Reading Notes: GeCCo

Guided Generation of Computational Cognitive Models

An Annotated Summary

Contents

1	Abstract & Overview	2
2	Motivation: The Bottleneck in Cognitive Modelling	2
3	LLMs as a Solution	2
4	The GeCCo Pipeline	3
4.1	Structure of Each Iteration	3
4.2	Fitting and Evaluation	3
4.3	Data Splits	3
4.4	Fail-safes	3
5	Metrics	3
6	Evaluation: Four Cognitive Domains	4
6.1	Experiment 1: Decision Making	4
6.2	Experiment 2: Learning	4
6.3	Experiment 3: Planning	4
6.4	Experiment 4: Working Memory	5
7	Control Experiments	5
7.1	Contribution of LLM Features	5
7.2	Prompt Ablation Study	5
7.3	Data Contamination	5
7.4	Ground Truth Recovery	5
7.5	Comparison with CENTAUR	6
8	Discussion	6
8.1	Key Strengths	6
8.2	Limitations & Future Directions	6
8.3	Broader Significance	6

Abstract & Overview

The GeCCo pipeline uses Large Language Models (LLMs) to automatically generate, fit, and iteratively refine computational cognitive models as Python functions. Given a task description, participant behavioural data, and a template function, GeCCo prompts an LLM to propose candidate models, fits those proposals to held-out data, and refines them based on feedback derived from predictive performance.

The approach was benchmarked across four cognitive domains—**decision making**, **learning**, **planning**, and **memory**—using three open-source LLMs of varying size, capacity, and training family. On all four human behavioural datasets, LLM-generated models consistently matched or outperformed the best domain-specific models from the literature, while remaining interpretable.

Code: <https://github.com/MilenaCCNlab/gecco.git>

Motivation: The Bottleneck in Cognitive Modelling

Computational cognitive models provide a formal framework for understanding human behaviour. In practice, researchers hand-craft these models, fit them to behavioural data, and iteratively refine them—a process that is:

- **Slow.** Translating a psychological theory into a working algorithm requires lengthy literature reviews and repeated trial-and-error coding cycles.
- **Skill-intensive.** Researchers must simultaneously be domain experts, theoreticians, and software engineers—a rare combination.
- **Biased by theoretical commitments.** Researchers tend to explore only the model classes they already know and favour, potentially missing superior explanations for the data.

This “streetlight effect” artificially shrinks the space of models considered, motivating an automated approach that is not anchored to any particular theoretical school.

LLMs as a Solution

Three specific LLM capabilities make them well-suited to cognitive model generation:

1. **Natural language flexibility.** LLMs can process task descriptions and behavioural data written in plain English, adapting across diverse experimental paradigms without a custom interface for each.
2. **In-context pattern recognition.** Drawing on broad pre-training knowledge, LLMs can identify structure in complex behavioural data and generate plausible psychological hypotheses.
3. **Code synthesis.** LLMs can translate hypotheses directly into executable Python functions, closing the loop from theory to testable model instantly.

These three abilities have already shown promise in automated statistical modelling (probabilistic program generation), and GeCCo applies the same logic to cognitive science.

The GeCCo Pipeline

Structure of Each Iteration

Each pipeline run consists of **10 sampling iterations**, repeated across **5 independent runs** per domain. On every iteration the LLM receives a structured prompt containing:

1. A natural language **task description**.
2. **Behavioural data** from a subset of participants (the *prompt* split).
3. **Instructions and guardrails** specifying the required Python function signature.
4. A **domain-specific template model** for syntactic guidance.
5. **Iterative feedback** (from iteration 2 onward): the current best-performing model and a list of previously used parameter names.

The LLM generates **three distinct cognitive models** with non-overlapping parameter sets.

Fitting and Evaluation

Generated models are parsed and converted to executable Python via `exec()`. Each model is then fit to a held-out *validation* dataset using `scipy.optimize.minimize` initialised from 20 random starting points. Model quality is scored with the **Bayesian Information Criterion (BIC)**—lower is better. The best-performing model is fed back into the next prompt.

Data Splits

Three separate data partitions prevent data leakage:

Split	Seen by LLM?	Purpose
Prompt data	Yes	Provides context for hypothesis generation
Validation data	No	Scores candidate models during the loop
Test data	No	Final evaluation against human baselines

Fail-safes

If a generated model contains syntax or runtime errors, the iteration restarts automatically—no human intervention required.

Metrics

- **BIC (Bayesian Information Criterion)**. Balances goodness of fit against model complexity by penalising the number of free parameters. When two models fit equally well, BIC favours the simpler one. The pipeline is metric-agnostic; AIC results are reported in Appendix I.
- **Exceedance Probability (EXP)**. Used in the final evaluation stage. Quantifies the posterior probability that a given model provides the most prevalent explanation

of observed behaviour across the population, translating raw BIC scores into an interpretable confidence measure.

Evaluation: Four Cognitive Domains

Four canonical paradigms were selected to form an *escalating complexity* ladder—from static decisions to rich interactions between memory systems.

Experiment 1: Decision Making

Task. Participants chose between two options defined by four features, each with a different validity (importance).

Baseline. The probabilistic Weighted Additive Decision (pWADD) model, which uses four separate parameters to weight the four features.

Result. The Llama-generated model achieved a substantially lower BIC (39.40 vs. 51.97; $p < .001$).

Insight. The LLM reduced four parameters to one *discount factor* that smoothly interpolates between three classic decision strategies—*Take the Best*, *Equal Weighting*, and *Weighted Additive*—mirroring complex Bayesian inference but arriving at the solution naturally.

Experiment 2: Learning

Task. A two-armed bandit with full feedback: after each trial participants also saw what the unchosen option would have paid, introducing regret and relief.

Baseline. A four-learning-rate variant of the Rescorla–Wagner model ($RW_{4\alpha}$), plus a “choice stickiness” (perseveration) parameter.

Result. BIC of 77.97 vs. 85.24; EXP = 0.97.

Insight. The LLM replaced the asymmetric learning rates with a clean separation between actual and counterfactual value traces (*fictive trace*), and replaced the perseveration hack with a psychologically motivated *forgetting parameter* that decays unchosen-option values over time—mirroring the limits of working memory.

Experiment 3: Planning

Task. The classic two-stage Daw task, in which choices lead to a second stage probabilistically, requiring forward planning.

Baseline. A Hybrid model combining model-free (SARSA) and model-based value iteration, averaged by a weighting parameter.

Result. BIC of 432.47 vs. 437.38; EXP = 0.85. (The BIC difference did not reach conventional significance at $p = .27$, but the population-level advantage is clear in EXP.)

Insight. The LLM unified the model-free/model-based dichotomy into a single system using two transition-dependent learning rates, and omitted the temporal discounting parameter, finding it unnecessary—aligning with recent theoretical calls to abandon strict dual-system accounts of planning.

Experiment 4: Working Memory

Task. Trial-and-error stimulus–response learning under varying cognitive load (3 vs. 6 associations held in mind simultaneously).

Baseline. The RL-WM model (Collins & Frank, 2012), in which a slow RL module and a fast but capacity-limited WM module operate independently, with a load-dependent mixing weight.

Result. BIC of 267.25 vs. 275.20; EXP = 0.99.

Insight. Contrary to the standard assumption that RL is immune to cognitive load, the LLM’s model assigned separate RL learning rates for low- and high-load conditions (`lr_low` / `lr_high`), discovering that load modulates the core learning signal—a conclusion consistent with neuroimaging evidence on reward prediction error signals.

Control Experiments

Contribution of LLM Features

A mixed-effects regression predicting BIC from *reasoning ability*, *model family*, and *model size* found that:

- Llama outperformed Qwen ($\beta = 5.624$, $p = .03$) despite Qwen having more parameters, suggesting training data and protocol matter more than raw scale.
- Reasoning ability showed a marginal advantage ($\beta = -0.266$, $p = .07$).

Prompt Ablation Study

Systematically removing each prompt component and refitting a mixed-effects model on post-ablation BIC scores revealed the following ranked importance:

Component	β	p-value
Iterative feedback	−17.040	< .05
Participant data	−12.836	< .05
Task description	−9.834	< .05
Code template	−9.359	< .05

Iterative feedback is the single most important driver of performance.

Data Contamination

The LogProber method was applied to Llama prompts across all four domains. Acceleration scores were well below the contamination threshold of 1.0 (range: 0.017–0.648), confirming that the LLM is reasoning about novel problems rather than recalling memorised solutions.

Ground Truth Recovery

On synthetic datasets with known generating processes, the LLM successfully reverse-engineered the true model in both learning (Rescorla–Wagner) and decision-making (Take the Best; Tallying) domains, validating the mathematical soundness of the approach.

Comparison with CENTAUR

CENTAUR is a large foundation model trained to predict human behaviour across cognitive tasks, serving as a proxy for the ceiling of explainable variance. GeCCo matched or exceeded CENTAUR’s negative log-likelihood across all four domains, while producing interpretable Python code rather than opaque model weights.

Discussion

Key Strengths

- **No fine-tuning required.** GeCCo relies entirely on in-context learning with off-the-shelf open-source LLMs, making it immediately accessible to researchers without specialised ML infrastructure.
- **Hybrid safety net.** The LLM acts only as a *proposal engine*; all scoring is performed by traditional offline optimisation tools, preventing the model from evaluating its own outputs.
- **Interpretability.** Unlike black-box foundation models such as CENTAUR, GeCCo outputs human-readable Python functions that scientists can inspect, debate, and build upon.
- **Speed.** Convergence takes hours rather than the days required by evolutionary LLM-search alternatives.

Limitations & Future Directions

- **Template bias.** The domain-specific template in the prompt may anchor the LLM toward a particular model class. An ablation showed this has the smallest performance impact, and having the LLM generate its own template produces equivalent results (Appendix J).
- **Purely numerical feedback.** The feedback loop currently only communicates BIC scores. Incorporating a second LLM as a “peer reviewer”—providing feedback on theoretical plausibility, parsimony, and code quality—could further reduce the need for human oversight.
- **Scope of domains.** The current evaluation covers four controlled laboratory paradigms. Extension to language processing, perception, vision, and naturalistic real-world datasets is a priority for future work.
- **High-dimensional models.** Generating models with many parameters or operating on high-dimensional data (e.g. vision) remains an open challenge.

Broader Significance

By removing the requirement to hand-craft code and by leveraging general-purpose open-source LLMs, GeCCo democratises access to sophisticated model discovery. Rather than spending months implementing and debugging theoretical models, researchers can use GeCCo to rapidly generate competing hypotheses and focus their effort on interpreting and extending the best ones—accelerating the pace of cognitive science.

Reference

Milena et al. (2024). *GeCCo: Guided Generation of Computational Cognitive Models*.
Available at: <https://github.com/MilenaCCNlab/gecco.git>